

FamilyBiz

Home Rent System



Developer Documentation

Version 1.0 | July 2026

[familybiz.online](#) | [api.familybiz.online](#)

Frontend		Backend
Vue 3 + Pinia	-->	Laravel 11 + Sanctum
familybiz.online	REST API	api.familybiz.online

Table of Contents

1. System Architecture	3
2. API Endpoints	4
3. Database Schema	6
4. Frontend Architecture	7
5. Feature Tracker	9
6. Deployment Guide	10
7. Testing	11

1. System Architecture

1.1 Architecture Overview

The FamilyBiz Home Rent System is a full-stack web application designed for property management. It follows a decoupled architecture with a RESTful API backend and a single-page application frontend.

Layer	Technology	Details
Backend	Laravel 11 (PHP 8.2+)	RESTful API with Sanctum token authentication
Frontend	Vue 3 + Vite + Pinia	SPA with role-based routing and persistent stores
Database	MySQL 8.0 / SQLite (testing)	12 core tables with foreign key constraints
Auth	Laravel Sanctum	Bearer token with localStorage persistence
Payment	ClickPesa Gateway	USSD push payments with webhook callbacks
Deployment	Laravel Cloud + Vercel	Auto-deploy from GitHub on push

1.2 Tech Stack

Technology	Purpose
PHP 8.2+	Server runtime for Laravel
Laravel 11	Backend framework with MVC architecture
Laravel Sanctum	API token authentication
MySQL 8.0	Production database
Vue 3 (Composition API)	Frontend framework
Vite	Build tool and dev server
Pinia	State management with persistence
Vue Router	Client-side routing with guards
Axios	HTTP client with interceptors
ClickPesa SDK	Mobile money payment gateway

1.3 Key Design Decisions

- **Token-based Authentication:** Uses Laravel Sanctum with Bearer tokens stored in localStorage. Tokens are attached via Axios request interceptor for all authenticated API calls.

- **Base64 Image Storage:** Room photos and room photos are stored as base64 strings directly in the database (LONGTEXT columns). Frontend converts File objects to base64 via FileReader before sending.
- **Role-based Routing:** Three roles (admin, landlord, tenant) with Vue Router navigation guards. Routes have meta.requiresAuth and meta.role fields. The getRole() helper checks is_super_admin > is_landlord > tenant.
- **30-Second Polling:** Dashboard data is refreshed every 30 seconds via setInterval for near-real-time updates on payment statuses, room occupancy, and announcements.
- **Manual Payment Confirmation:** Landlords can manually confirm tenant payments. The system also supports ClickPesa automatic confirmation via webhook callback.

1.4 Project Structure

```

HomeRentSystemFrontend/src/
components/
auth/ -- LandLordDashboard, TenantDashboard, SuperAdminDashboard
auth/edits/ -- RoomEdit, PaymentEdit, PaymentMethodEdit, AnnouncementEdit
Login.vue, Register.vue, ForgotPassword.vue
composables/
api.js -- Axios instance with baseURL and interceptors
usePagination.js -- Client-side pagination composable
guest/register.js -- Registration logic
router/index.js -- Route definitions and navigation guards
stores/
auth.js, room.js, payment.js, admin.js, announcement.js
comment.js, rules.js, paymentMethod.js, criticalRemark.js
latePaymentReason.js, entertainment/football.js
views/
HomeView.vue, AboutView.vue

```

2. API Endpoints

All endpoints are prefixed with `/api`. Protected routes require `Authorization: Bearer '{{token}}'` header.

Public Routes (No Authentication)

Method	Path	Middleware	Description
GET	<code>/api/health</code>	None	API health check with version info
POST	<code>/api/user/create</code>	None	Register new user account
POST	<code>/api/user/auth</code>	None	Login and receive token
POST	<code>/api/user/forgot-password</code>	None	Request password reset link
POST	<code>/api/payment/callback</code>	None	ClickPesa webhook callback

Football Routes (Public)

Method	Path	Middleware	Description
GET	<code>/api/football/live</code>	None	Fetch live match scores
GET	<code>/api/football/fixtures</code>	None	Fetch match fixtures
GET	<code>/api/football/standings</code>	None	Fetch league standings
GET	<code>/api/football/leagues</code>	None	Fetch available leagues
GET	<code>/api/football/match/{fixtureId}</code>	None	Fetch specific match details
GET	<code>/api/football/team/{teamId}</code>	None	Fetch team details
GET	<code>/api/football/scorers</code>	None	Fetch top scorers
GET	<code>/api/football/test</code>	None	Test API connection

Protected Routes (auth:sanctum)

Authentication

Method	Path	Middleware	Description
GET	<code>/api/user/fetch</code>	sanctum	Fetch authenticated user profile
POST	<code>/api/user/logout</code>	sanctum	Revoke current token

User Management

Method	Path	Middleware	Description
GET	/api/user/show/{id}	sanctum	Get user by ID
PATCH	/api/user/update/{id}	sanctum	Update user details
PATCH	/api/user/update/phone/{id}	sanctum	Update phone number
PATCH	/api/user/update/profile/{id}	sanctum	Update user profile
DELETE	/api/user/delete/{id}	sanctum	Delete user account
POST	/api/room/select/{roomId}	sanctum	Tenant selects a room
PATCH	/api/payment/confirm/{paymentId}	sanctum	Landlord confirms payment
GET	/api/payment/unconfirmed	sanctum	Get unconfirmed payments
PATCH	/api/payment/cancel/{paymentId}	sanctum	Tenant cancels payment
PATCH	/api/payment/reject/{paymentId}	sanctum	Landlord rejects payment

Room Management

Method	Path	Middleware	Description
GET	/api/room/fetch	sanctum	List all rooms with stats
GET	/api/room/show/{id}	sanctum	Get room by ID
POST	/api/room/create	sanctum	Create new room
PATCH	/api/room/update/{id}	sanctum	Update room details
PATCH	/api/room/update/status/{id}	sanctum	Update room status
DELETE	/api/room/delete/{id}	sanctum	Delete room

Room Photos

Method	Path	Middleware	Description
GET	/api/room-photos/fetch	sanctum	List all room photos
GET	/api/room-photos/show/{id}	sanctum	Get photo by ID
POST	/api/room-photos/create	sanctum	Upload room photo
DELETE	/api/room-photos/delete/{id}	sanctum	Delete room photo

Payments

Method	Path	Middleware	Description
GET	/api/payment/fetch	sanctum	List all payments
GET	/api/payment/show/{id}	sanctum	Get payment by ID
POST	/api/payment/create	sanctum	Create payment (initiates ClickPesa)
PATCH	/api/payment/update/{id}	sanctum	Update payment details
DELETE	/api/payment/delete/{id}	sanctum	Delete payment
GET	/api/payment/status/{paymentId}	sanctum	Check payment status

Payment Methods

Method	Path	Middleware	Description
GET	/api/method/fetch	sanctum	List all payment methods

Method	Path	Middleware	Description
GET	/api/method/show/{id}	sanctum	Get method by ID
POST	/api/method/create	sanctum	Create payment method
PATCH	/api/method/update/{id}	sanctum	Update payment method
DELETE	/api/method/delete/{id}	sanctum	Delete payment method

Critical Remarks

Method	Path	Middleware	Description
GET	/api/remarks/fetch	sanctum	List all critical remarks
GET	/api/remarks/show/{id}	sanctum	Get remark by ID
POST	/api/remarks/create	sanctum	Create critical remark
PATCH	/api/remarks/update/{id}	sanctum	Update remark
DELETE	/api/remarks/delete/{id}	sanctum	Delete remark

Late Payment Reasons

Method	Path	Middleware	Description
GET	/api/reasons/fetch	sanctum	List all late payment reasons
GET	/api/reasons/show/{id}	sanctum	Get reason by ID
POST	/api/reasons/create	sanctum	Create late payment reason
PATCH	/api/reasons/update/{id}	sanctum	Update reason
DELETE	/api/reasons/delete/{id}	sanctum	Delete reason

Announcements

Method	Path	Middleware	Description
GET	/api/announcements/fetch	sanctum	List all announcements
GET	/api/announcements/show/{id}	sanctum	Get announcement by ID
POST	/api/announcements/create	sanctum	Create announcement
PATCH	/api/announcements/update/{id}	sanctum	Update announcement
DELETE	/api/announcements/delete/{id}	sanctum	Delete announcement

Comments

Method	Path	Middleware	Description
GET	/api/comments/fetch	sanctum	List all comments
GET	/api/comments/show/{id}	sanctum	Get comment by ID

Method	Path	Middleware	Description
POST	/api/comments/create	sanctum	Create comment with rating
PATCH	/api/comments/update/{id}	sanctum	Update comment
DELETE	/api/comments/delete/{id}	sanctum	Delete comment

Rules

Method	Path	Middleware	Description
GET	/api/rule/fetch	sanctum	List all rules
GET	/api/rule/show/{id}	sanctum	Get rule by ID
POST	/api/rule/create	sanctum	Create rule
PATCH	/api/rule/update/{id}	sanctum	Update rule
DELETE	/api/rule/delete/{id}	sanctum	Delete rule

Admin Routes (admin middleware)

Method	Path	Middleware	Description
GET	/api/admin/landlords	sanctum+admin	List all landlords (paginated)
GET	/api/admin/tenants	sanctum+admin	List all tenants (paginated)
PATCH	/api/admin/user/{userId}/reset-password	sanctum+admin	Reset user password
PATCH	/api/admin/landlord/{userId}/toggle-status	sanctum+admin	Activate/deactivate landlord
PATCH	/api/admin/user/{userId}/toggle-status	sanctum+admin	Activate/deactivate user

3. Database Schema

The system uses 12 core tables. Below is each table with its columns, types, and constraints.

users

Column	Type	Constraints
id	bigint (PK)	AUTO_INCREMENT
room_id	bigint (FK)	NULLABLE -> rooms.id
is_landlord	boolean	DEFAULT false
is_super_admin	boolean	DEFAULT false
is_active	boolean	DEFAULT true
last_name	varchar	REQUIRED
phone_number	varchar	REQUIRED
email	varchar	UNIQUE, REQUIRED
password	varchar	HASHED, REQUIRED
email_verified_at	timestamp	NULLABLE
remember_token	varchar	REMEMBER_ME
created_at / updated_at	timestamps	AUTO

rooms

Column	Type	Constraints
id	bigint (PK)	AUTO_INCREMENT
user_id	bigint (FK)	NULLABLE -> users.id, CASCADE
room_price	float(8,2)	DEFAULT 0
room_number	varchar	REQUIRED
status	varchar	Available Occupied Maintenance
type	varchar	DEFAULT 'available'
photo	longtext	NULLABLE (base64 encoded)
created_at / updated_at	timestamps	AUTO

payments

Column	Type	Constraints
id	bigint (PK)	AUTO_INCREMENT
user_id	bigint (FK)	REQUIRED -> users.id, CASCADE
room_id	bigint (FK)	REQUIRED -> rooms.id, CASCADE
month	integer (unsigned)	REQUIRED (1-12)
year	integer (unsigned)	REQUIRED
amount	integer (unsigned)	DEFAULT 0
status	string	pending paid unpaid failed cancelled
due_date	date	NULLABLE
paid_at	timestamp	NULLABLE
payment_method	string	DEFAULT 'clickpesa'
clickpesa_transaction_id	string	NULLABLE
clickpesa_response	text	NULLABLE
confirmed_by_landlord	boolean	DEFAULT false
room_selected	boolean	DEFAULT false
confirmation_message	text	NULLABLE
created_at / updated_at	timestamps	AUTO

payment_methods

Column	Type	Constraints
id	bigint (PK)	AUTO_INCREMENT
user_id	bigint (FK)	REQUIRED -> users.id, CASCADE
airtel_money_number	varchar	NULLABLE
m_pesa_number	varchar	NULLABLE
mixx_by_yas_number	varchar	NULLABLE
halopesa_number	varchar	NULLABLE
nmb_account_number	varchar	NULLABLE
crdb_account_number	varchar	NULLABLE
nbc_account_number	varchar	NULLABLE
created_at / updated_at	timestamps	AUTO

room_photos

Column	Type	Constraints
id	bigint (PK)	AUTO_INCREMENT
room_id	bigint (FK)	REQUIRED -> rooms.id, CASCADE
photo	longtext	NULLABLE (base64 encoded)
created_at / updated_at	timestamps	AUTO

critical_remarks

Column	Type	Constraints
id	bigint (PK)	AUTO_INCREMENT
user_id	bigint (FK)	REQUIRED -> users.id, CASCADE
reason_text	longtext	REQUIRED
type	varchar	REQUIRED
active	boolean	DEFAULT false
created_at / updated_at	timestamps	AUTO

late_payment_reasons

Column	Type	Constraints
id	bigint (PK)	AUTO_INCREMENT
user_id	bigint (FK)	REQUIRED -> users.id, CASCADE
payment_id	bigint (FK)	REQUIRED -> payments.id, CASCADE
reason_text	longtext	REQUIRED
created_at / updated_at	timestamps	AUTO

rules

Column	Type	Constraints
id	bigint (PK)	AUTO_INCREMENT
title	varchar	REQUIRED
description	longtext	REQUIRED
type	varchar	REQUIRED
created_at / updated_at	timestamps	AUTO

announcements

Column	Type	Constraints
id	bigint (PK)	AUTO_INCREMENT
user_id	bigint (FK)	REQUIRED -> users.id, CASCADE
title	varchar	REQUIRED
message	longtext	REQUIRED
created_at / updated_at	timestamps	AUTO

comments

Column	Type	Constraints
id	bigint (PK)	AUTO_INCREMENT
user_id	bigint (FK)	REQUIRED -> users.id, CASCADE

Column	Type	Constraints
comment	text	REQUIRED
rating	tinyint (unsigned)	1-5 STARS
created_at / updated_at	timestamps	AUTO

houses

Column	Type	Constraints
id	bigint (PK)	AUTO_INCREMENT
house_price	float(8,2)	DEFAULT 0
house_name	varchar	REQUIRED
total_rooms	integer	REQUIRED
address	varchar	REQUIRED
description	longtext	REQUIRED
created_at / updated_at	timestamps	AUTO

room_selections

Column	Type	Constraints
id	bigint (PK)	AUTO_INCREMENT
user_id	bigint (FK)	REQUIRED -> users.id, CASCADE
room_id	bigint (FK)	REQUIRED -> rooms.id, CASCADE
created_at / updated_at	timestamps	AUTO

personal_access_tokens

Column	Type	Constraints
id	bigint (PK)	AUTO_INCREMENT
tokenable_type + tokenable_id	morphs	POLYMORPHIC
name	text	REQUIRED
token	varchar(64)	UNIQUE

Column	Type	Constraints
abilities	text	NULLABLE
last_used_at	timestamp	NULLABLE
expires_at	timestamp	NULLABLE, INDEXED
created_at / updated_at	timestamps	AUTO

4. Frontend Architecture

4.1 Component Tree

```
App.vue
HomeView.vue -- Landing page (public)
AboutView.vue -- About page (public, lazy-loaded)
Login.vue -- Login form (guest-only)
Register.vue -- Registration form (guest-only)
ForgotPassword.vue -- Password reset form (guest-only)
LandLordDashboard.vue -- Landlord main dashboard
RoomEdit.vue -- Edit room (/:id)
PaymentEdit.vue -- Edit payment (/:id)
PaymentMethodEdit.vue -- Edit payment method (/:id)
AnnouncementEdit.vue -- Edit announcement (/:id)
TenantDashboard.vue -- Tenant main dashboard
SuperAdminDashboard.vue -- Admin panel
```

4.2 Pinia Stores

Store	Key Actions
auth	login, fetchUser, fetchUsers, logout, updatePhoneNumber, updateProfile, requestPasswordReset
room	registerRoom, fetchRooms, loadRoomForEdit, updateRoom, updateRoomStatus, deleteRoom
payment	fetchPayments, fetchPayment, registerPayment, updatePayments, deletePayment, selectRoom, confirmPayment, fetchUnconfirmedPayments, cancelPayment, rejectPayment
admin	fetchLandlords, fetchTenants, resetPassword, toggleUserStatus
announcement	fetchAnnouncements, registerAnnouncement, deleteAnnouncement, loadAnnouncementForEdit, updateAnnouncement
comment	fetchComments, registerComments, deleteComment
rule	fetchRules, registerRules
paymentMethod	fetchPaymentMethods, registerPaymentMethods, loadPaymentMethodForEdit, updatePaymentMethod, deletePaymentMethod
criticalRemark	fetchCriticalRemarks, registerCriticalRemarks, loadCriticalRemarkForEdit, updateCriticalRemark, deleteCriticalRemark
latePaymentReason	fetchLatePaymentReasons, registerLatePaymentReasons
football	fetchLiveMatches, fetchFixtures, fetchStandings, fetchMatchDetails, fetchTeamDetails, fetchTopScorers, clearData

4.3 Composables

Composable	Description
api.js	Axios instance with baseURL=https://api.familybiz.online, Bearer token interceptor, FormData handling
usePagination.js	Client-side pagination: currentPage, totalPages, paginatedData, goToPage, nextPage, prevPage
guest/register.js	Registration form logic and validation

4.4 Authentication Flow

- 1. User submits credentials to POST /api/user/auth
- 2. Backend validates and returns {user, token}
- 3. Frontend stores token in localStorage('auth_token')
- 4. Pinia auth store sets user = response.data.user
- 5. Axios interceptor attaches Bearer token to all requests
- 6. Router guard calls auth.fetchUser() on navigation
- 7. fetchUser() calls GET /api/user/fetch to validate token
- 8. On logout, token removed from localStorage and user = null

4.5 Role-Based Routing

Route	Condition	Component	Notes
/admin	is_super_admin=true	SuperAdminDashboard.vue	Full system management
/landlord	is_landlord=true	LandLordDashboard.vue	Room & payment management
/tenant	default role	TenantDashboard.vue	Room view & payment status
/	requiresGuest	HomeView.vue	Public landing page
/login	requiresGuest	Login.vue	Guest-only redirect if logged in
/register	requiresGuest	Register.vue	Guest-only redirect if logged in

Navigation Guard Logic: The router.beforeEach hook checks: (1) requiresAuth without user -> redirect to /login; (2) requiresGuest with user -> redirect to role default; (3) role mismatch -> redirect to correct role dashboard.

4.6 Image Handling

Room photos are stored as base64 strings in LONGTEXT columns. The frontend uses `FileReader.readAsDataURL()` to convert File objects before API submission. The `RoomController` adds a `photo_url` field to each room response for direct rendering in ``.

4.7 Internationalization

The system uses locale-based routing (`/en/...`, `/sw/...`). The API uses `App::setLocale()` to switch language per request. Frontend locale is managed via router URL prefixes and the API automatically responds in the requested language.

5. Feature Tracker

Comprehensive list of all implemented features with status and component mapping.

Feature	Status	Backend Component	Frontend Component	Notes
User Registration	✓ DONE	UserController@store	Register.vue	Email + phone + password
User Login (Sanctum)	✓ DONE	AuthController@login	Login.vue	Token-based authentication
Password Reset	✓ DONE	AuthController@forgotPassword	ForgotPassword.vue	Email reset link
Room CRUD	✓ DONE	RoomController	LandLordDashboard	Create, read, update, delete rooms
Room Status Toggle	✓ DONE	RoomController@updateRoomStatus	LandLordDashboard	Available/Occupied/Maintenance
Room Photo Upload (Base64)	✓ DONE	RoomController@store	RoomEdit.vue	Base64 encoded images
Room Photo Gallery	✓ DONE	RoomPhotoController	LandLordDashboard	Multiple photos per room
Payment Creation (ClickPesa)	✓ DONE	PaymentController@store	PaymentEdit.vue	USSD push payments
Payment Status Tracking	✓ DONE	PaymentController@checkStatus	TenantDashboard	Real-time status check
Payment Confirmation (Landlord)	✓ DONE	UserController@confirmPayment	LandLordDashboard	Manual confirmation flow
Payment Rejection (Landlord)	✓ DONE	UserController@rejectPayment	LandLordDashboard	Reject pending payments
Payment Cancellation (Tenant)	✓ DONE	UserController@cancelPayment	TenantDashboard	Cancel pending payments
Room Selection Flow	✓ DONE	UserController@selectRoom	TenantDashboard	Select room + auto-create payment
Unconfirmed Payments View	✓ DONE	UserController@getUnconfirmedPayments	LandLordDashboard	Manual payment queue
Payment Methods Management	✓ DONE	PaymentMethodController	LandLordDashboard	7 mobile money/bank options

Feature	Status	Backend Component	Frontend Component	Notes
Announcements CRUD	✓ DONE	AnnouncementController	LandLordDashboard	Title + message system
Comments & Ratings	✓ DONE	CommentController	TenantDashboard	1-5 star rating system
Critical Remarks	✓ DONE	CriticalRemarkController	LandLordDashboard	Tenant behavior tracking
Late Payment Reasons	✓ DONE	LatePaymentReasonController	TenantDashboard	Reason text per payment
Rules Management	✓ DONE	RuleController	LandLordDashboard	Title + description + type
Super Admin Panel	✓ DONE	AdminController	SuperAdminDashboard.vue	User/landlord management
Admin Password Reset	✓ DONE	AdminController@resetUserPassword	SuperAdminDashboard.vue	Force password change
Admin Toggle User Status	✓ DONE	AdminController@toggleUserStatus	SuperAdminDashboard.vue	Activate/deactivate users
Admin Landlord Toggle	✓ DONE	AdminController@toggleLandlordStatus	SuperAdminDashboard.vue	Activate/deactivate landlords
Pagination (Client-Side)	✓ DONE	usePagination.js	All dashboards	15 items per page
Role-Based Routing	✓ DONE	router/beforeEach	router/index.js	3 roles with guards
Persistent Auth State	✓ DONE	pinia-persist	auth.js	User persisted to localStorage
Football Live Scores	✓ DONE	FootballController	football.js store	Live match data
Football Fixtures	✓ DONE	FootballController	football.js store	Date-filtered fixtures
Football Standings	✓ DONE	FootballController	football.js store	League table data
ClickPesa Webhook	✓ DONE	PaymentController@callback	api.php (public)	Auto-confirm payments
API Health Check	✓ DONE	Route::get('/health')	api.php (public)	Version + environment info
Search & Filter	✓ DONE	All controllers	All dashboards	Paginated search with params

Feature	Status	Backend Component	Frontend Component	Notes
30-Second Polling	✓ DONE	setInterval	All dashboards	Near-real-time data refresh
Forgot Password Flow	✓ DONE	AuthController	ForgotPassword.vue	Email-based reset

6. Deployment Guide

6.1 Backend (Laravel Cloud)

The Laravel API is deployed on **Laravel Cloud** with automatic deployments triggered by GitHub pushes.

- **Repository:** GitHub repository connected to Laravel Cloud
- **Auto-Deploy:** Push to main branch triggers automatic deployment
- **Database:** MySQL 8.0 provisioned via Laravel Cloud
- **Environment:** Production environment with queued jobs
- **Domain:** api.familybiz.online (configured in Laravel Cloud DNS)

6.2 Environment Variables

Variable	Value	Description
APP_NAME	FamilyBiz	Application name
APP_ENV	production	Environment
APP_KEY	base64:...	Encryption key
APP_URL	https://api.familybiz.online	Backend URL
DB_CONNECTION	mysql	Database driver
DB_HOST	Laravel Cloud managed	Database host
DB_PORT	3306	Database port
DB_DATABASE	home_rent_system	Database name
SANCTUM_STATEFUL_DOMAINS	familybiz.online	CORS domains
CLICKPESA_API_KEY	***	Payment gateway key
CLICKPESA_CLIENT_ID	***	Payment gateway client ID
MAIL_MAILER	smtp	Password reset emails

6.3 Frontend (Vercel / Static)

The Vue 3 frontend is a static SPA deployed to Vercel or similar platform.

- **Build Command:** npm run build
- **Output Directory:** dist

- **Base URL:** familybiz.online
- **API Base URL:** https://api.familybiz.online (configured in api.js)
- **Environment:** VITE_API_URL for dynamic API base

6.4 CLI Commands

Command	Description
npm install	Install frontend dependencies
npm run dev	Start Vite dev server (localhost:5173)
npm run build	Build production bundle
php artisan migrate	Run database migrations
php artisan test	Run backend test suite
php artisan serve	Start local Laravel server (localhost:8000)
composer install	Install backend PHP dependencies

6.5 Repository URLs

Component	URL	Notes
Backend API	github.com/.../HomeRentSystemApi	Laravel 11 application
Frontend SPA	github.com/.../HomeRentSystemFrontend	Vue 3 application
Production API	api.familybiz.online	Laravel Cloud hosted
Production Frontend	familybiz.online	Vercel / static hosting

7. Testing

7.1 Test Results

89	Tests	242	Assertions	PASSED
----	-------	-----	------------	--------

7.2 How to Run Tests

```
cd HomeRentSystemApi
php artisan test
# Run specific test file
php artisan test --filter=AuthTest
php artisan test --filter=RoomControllerTest
```

7.3 Test Files & Coverage

Test File	Component	Coverage
AuthTest.php	Authentication endpoints	Login, logout, register, forgot-password
RoomControllerTest.php	Room CRUD operations	Create, read, update, delete, status toggle
PaymentControllerTest.php	Payment operations	Create, update, delete, status check, callback
UserControllerTest.php	User management	Profile update, phone update, user listing
AnnouncementControllerTest.php	Announcements	Full CRUD operations
CommentControllerTest.php	Comments & ratings	Create, list, delete comments
RuleControllerTest.php	Rules management	Create, list rules
CriticalRemarkControllerTest.php	Critical remarks	Full CRUD operations
LatePaymentReasonControllerTest.php	Late reasons	Create and list reasons
PaymentMethodControllerTest.php	Payment methods	7 payment method types
AdminControllerTest.php	Admin operations	List users, toggle status, reset password
RoomPhotoControllerTest.php	Room photos	Upload, list, delete photos
TestCase.php	Base test setup	Factories, auth helpers, assertions

7.4 Test Architecture

Tests use Laravel's built-in testing framework with RefreshDatabase trait for isolated test runs. Each test creates fresh database state via model factories. Authentication tests use Sanctum's actingAs() helper.

- **Database Isolation:** RefreshDatabase trait ensures each test starts with clean state
- **Model Factories:** User, Room, Payment factories for realistic test data
- **HTTP Testing:** Laravel's HTTP testing methods (get, post, patch, delete)
- **JSON Assertions:** assertJson, assertJsonStructure for API response validation
- **Status Code Assertions:** assertOk, assertCreated, assertNotFound, etc.
- **Authentication Testing:** Sanctum actingAs for protected route testing